PREDICTIVE ANALYTICS USING HISTORICAL DATA

Python project solution — forecasting future monthly sales

Objective: Build a predictive model from historical data, evaluate its accuracy and forecast future trends. This solution uses monthly sales as an example and can be adapted to other numeric time-series data.

1. Introduction

Predictive analytics uses historical data, statistics and machine learning to estimate future outcomes. Businesses use it for sales forecasting, inventory planning, budgeting, demand estimation and decision-making.

2. Objectives

• Clean and prepare historical data.

• Visualise past values and identify trends.

• Create time-based features.

• Train a regression model and evaluate it on later observations.

• Forecast the next 12 months and save the results.

3. Dataset and Tools

The program reads **historical_data.csv** with two columns: **Date** and **Sales**, one row per month. If the file is missing, the program creates illustrative synthetic data so it can run as a demonstration. Synthetic results must not be presented as real business data. Libraries: pandas, NumPy, Matplotlib and scikit-learn.

4. Methodology

Data preparation: Convert dates and sales values to the correct types, remove invalid rows, sort dates and combine duplicate months. **Feature engineering:** Create a time index and sine/cosine features for month-of-year seasonality. **Training:** Train a Random Forest regression model using the earliest 80% of observations. **Evaluation:** Test on the latest 20% using MAE and RMSE. **Forecast:** Refit on all available data and predict 12 future months.

5. Python Implementation

```
# Save as predictive_analytics.py
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

FILE = "historical_data.csv"
HORIZON = 12

# Create sample data only if no real dataset is supplied
if not os.path.exists(FILE):
    rng = np.random.default_rng(42)
    dates = pd.date_range("2018-01-01", periods=72, freq="MS")
    m = np.array([d.month for d in dates])
    sales = (1000 + np.arange(72)*8
             + 100*np.sin(2*np.pi*(m-1)/12)
             + rng.normal(0, 35, 72))
    pd.DataFrame({"Date": dates, "Sales": sales}).to_csv(FILE, index=False)
    print("Created illustrative sample data.")

# Load and clean
df = pd.read_csv(FILE)
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")
df = df.dropna(subset=["Date", "Sales"]).sort_values("Date")
df["Date"] = df["Date"].dt.to_period("M").dt.to_timestamp()
df = df.groupby("Date", as_index=False)["Sales"].mean()
if len(df) < 18:
    raise ValueError("Use at least 18 monthly observations.")

def features(dates, start_date):
    dates = pd.Series(pd.to_datetime(dates))
    idx = ((dates.dt.year-start_date.year)*12
           + dates.dt.month-start_date.month)
    month = dates.dt.month
    return pd.DataFrame({
        "time_index": idx,
        "month_sin": np.sin(2*np.pi*month/12),
```

```
        "month_cos": np.cos(2*np.pi*month/12)
    })

start = df["Date"].min()
X = features(df["Date"], start)
y = df["Sales"].to_numpy()

# Chronological split: most recent 20% is held out for testing
split = int(len(df)*0.8)
model = RandomForestRegressor(
    n_estimators=300, random_state=42, min_samples_leaf=2
)
model.fit(X.iloc[:split], y[:split])
pred = model.predict(X.iloc[split:])
mae = mean_absolute_error(y[split:], pred)
rmse = np.sqrt(mean_squared_error(y[split:], pred))
print(f"Test MAE: {mae:.2f}")
print(f"Test RMSE: {rmse:.2f}")

plt.figure(figsize=(9, 4))
plt.plot(df["Date"], y, marker="o")
plt.title("Historical Monthly Sales")
plt.xlabel("Date"); plt.ylabel("Sales")
plt.tight_layout(); plt.savefig("historical_sales.png", dpi=150)
plt.show()

plt.figure(figsize=(9, 4))
plt.plot(df["Date"].iloc[split:], y[split:], marker="o", label="Actual")
plt.plot(df["Date"].iloc[split:], pred, marker="o", label="Predicted")
plt.title("Actual vs Predicted (Test Period)")
plt.xlabel("Date"); plt.ylabel("Sales"); plt.legend()
plt.tight_layout(); plt.savefig("actual_vs_predicted.png", dpi=150)
plt.show()

# Refit on all observations and forecast next 12 months
final_model = RandomForestRegressor(
    n_estimators=300, random_state=42, min_samples_leaf=2
)
final_model.fit(X, y)
future_dates = pd.date_range(
    df["Date"].max() + pd.offsets.MonthBegin(1),
    periods=HORIZON, freq="MS"
)
future = pd.DataFrame({
    "Date": future_dates,
    "Forecast_Sales": final_model.predict(features(future_dates, start))
})
future.to_csv("sales_forecast.csv", index=False)
print(future.to_string(index=False))

plt.figure(figsize=(9, 4))
plt.plot(df["Date"], y, label="Historical")
plt.plot(future["Date"], future["Forecast_Sales"],
         marker="o", linestyle="--", label="Forecast")
plt.title("12-Month Sales Forecast")
plt.xlabel("Date"); plt.ylabel("Sales"); plt.legend()
plt.tight_layout(); plt.savefig("sales_forecast.png", dpi=150)
plt.show()
print("Saved charts and sales_forecast.csv")
```

6. How to Run

1. Install Python 3. In a terminal run: **pip install pandas numpy matplotlib scikit-learn**.
2. Save the code as **predictive_analytics.py**.
3. Optional: place your own **historical_data.csv** in the same folder with Date and Sales columns.
4. Run **python predictive_analytics.py**.

7. Evaluation Metrics

MAE (Mean Absolute Error): average absolute difference between actual and predicted values. Lower is better. **RMSE (Root Mean Squared Error):** square root of the average squared error; it penalises large mistakes more strongly. Both use the same units as Sales. Results depend on the dataset and will be printed when the code runs.

8. Expected Outputs

Output file	Purpose
historical_sales.png	Visualises the historical trend.
actual_vs_predicted.png	Compares actual and predicted values in the test period.
sales_forecast.png	Shows historical data and the next 12-month forecast.
sales_forecast.csv	Contains future dates and predicted sales.

9. Applications

Sales and demand forecasting, inventory management, revenue planning, staffing, and trend analysis.

10. Limitations

• Forecast quality depends on data quality and the number of observations.

• Unexpected events, price changes and promotions may not be captured.

• Random Forest may not extrapolate long-term trends well; compare other models for important use cases.

• Predictions are estimates and are not guaranteed future outcomes.

11. Future Scope

Add variables such as promotions, holidays and prices; compare regression with dedicated time-series models; update forecasts as new data arrives; and create a Power BI dashboard.

12. Conclusion

This project demonstrates a complete predictive analytics workflow: data cleaning, feature engineering, model training, chronological evaluation and future forecasting. The results can support planning when interpreted alongside model error and domain knowledge.

Submission checklist

■ Add your name, roll number and college if required.
■ Use a real dataset where possible and state its source.
■ Add screenshots of the generated charts and printed MAE/RMSE.
■ Upload the PDF, code and outputs to GitHub or Drive, then submit a shareable link.